01 · Project Information — Science Agent Coworker Hajimi

> Submission copy for *Global AI4Science Hackathon 2026*. Track: **AI Agent**. This Markdown is the source of truth; `node scripts/build-submission-pdf.mjs` renders the PDF (`01-project-info.en.pdf`) that is submitted alongside it.

Project name

Science Agent Coworker Hajimi — a research agent that grows three reusable libraries from the papers it reads.

(One-line pitch: reading a paper should leave something behind, not just a chat log.)

Summary

Science Agent Coworker Hajimi is a research agent that turns papers into **three kinds of reusable assets** instead of answering questions and forgetting them:

Library	What it accumulates	The question it answers
Knowledge library	Atomic claims, each anchored to a verbatim quote and a verified page number	"What is known, and where exactly was it said?"
Tool library	Executable tools extracted from the methods described in papers	"How is it done, and can I run it?"
Skill library	Seven-section reviewable skill files distilled across papers	"What should I do, and what should I avoid?"

The next time a similar task appears, the agent works from these assets instead of re-reading the literature. The libraries compound; the agent gets measurably more useful with every paper added.

The scientific problem we address

Researchers do not suffer from a shortage of papers. They suffer from **knowledge that does not survive the reading**.

Three specific failures, all of which we treat as correctness problems rather than UX problems:

1. **Claims lose their provenance.** A graduate student reads twenty papers before a group meeting. Six weeks later they remember the conclusion but not which paper, which section, or which page. Re-deriving it costs hours. In a general-purpose LLM chat, the conclusion is reproduced fluently and confidently — with no way to check it. For research use, an unverifiable claim is worse than no claim.
2. **Methods are described but never become runnable.** Papers report a procedure in prose. Making it executable is manual work that is thrown away after one project, so the same method is re-implemented by

every group that needs it.

3. **Tacit judgement does not transfer.** The part that actually determines whether a method works — thresholds, preconditions, common failure modes — lives in the heads of experienced researchers. New members rediscover it by failing.

Existing tools address retrieval (search over papers) or summarization (a longer answer). Neither addresses **accumulation with provenance**: persistent, auditable assets that a later task can rely on.

What the system does

```
PDF ⟶ parsed document (paragraphs + page boundaries)
     ⟶ knowledge library   : 80 atomic notes from 4 papers, every note carrying
                             a verbatim quote + a page number reverse-located from that quote
     ⟶ tool library        : methods extracted as specs, rendered into runnable TypeScript
                             tools, each shipped with a smoke test
     ⟶ skill library       : seven-section skill files, drafts held until human review
```

Two design decisions matter for evaluation:

- **Page numbers are never taken from the model.** Across two instrumented reading runs (ReAct, arXiv 2210.03629, 20 claims; Transformer, arXiv 1706.03762, 18 claims), **1 of 38** self-reported page numbers was right — the rest were off by +1 (28 cases), +2 (7), −4 (1) or −3 (1), so no arithmetic correction exists. The quotes themselves were locatable **38/38**. We report this as a **design-settling observation, not an error rate**: it settles one design question rather than estimating a population — a page number may never be taken on trust. Every page in the knowledge library is therefore reverse-located from the verbatim quote against the PDF's own page boundaries; **99.7%–100%** locate rate over 80 notes.
- **Tools are grown from papers, not hand-written.** The agent extracts a method specification with an evidence page, and a deterministic template renders it into a tool. Five tools are active today; where no verified implementation exists, the system degrades to a parameter template for a human to fill in — it never silently emits plausible-looking code.

SciencePal is not cosmetic in this pipeline. The main loop calls a SciencePal agent as one of its tool nodes (`sciencepal_query`) for deep literature work: SciencePal performs the close reading and returns claims with verbatim quotes; our system then verifies and anchors those quotes to real pages and governs what is allowed into the knowledge base. We maintain two custom SciencePal agents — "Research Distiller" (`9e70403e-…`), which executed every measured run committed to this repository, and "Paper Close Reader" (`ae5aaadf-…`), the close-reading configuration — one on each of two members' accounts. Run attribution is read from the `agentId` field of `fixtures/runs/<runId>/manifest.json`.

The rules state that using SciencePal *only* for writing copy, polishing text or producing slides does not count as valid use. Our claim therefore does not rest on text assistance of any kind: SciencePal is a runtime component of the product's core loop, and the artefacts it produces are machine-consumed by our pipeline rather than pasted into a document.

Target users and usage scenarios

Primary users

User	Situation	What Science Agent Coworker Hajimi gives them
Graduate students / early-career researchers	Entering a new subfield; reading 20–50 papers	A citation-grade knowledge base they can query months later, with page-level provenance
Lab PIs and senior researchers	Onboarding new members; writing related work	A reusable method and judgement library instead of one-off verbal explanations
Systematic-review and meta-analysis teams	Screening and extracting from many papers	Auditable extraction where every statement maps back to a page

Concrete scenarios

1. **Before a group meeting.** Drop the week's papers into the browser. Twenty atomic claims per paper land in the knowledge library, each with its source page. Later, asking "which paper reported this result, and on which page?" returns an answer that can be checked in seconds.
2. **Reusing a method.** A method described in a paper becomes a runnable tool with a smoke test. The next project calls the tool instead of re-implementing the procedure.
3. **Handing over expertise.** A skill file records when to use a method, what the acceptance criteria are, and how it typically fails. Drafts stay in a review queue and only become active after a human approves them.
4. **Cross-paper consensus.** A skill section tagged `evidence:3` means three independent papers support it; single-paper preferences are labelled as such. The agent is required to say "only seen in individual studies" rather than presenting one lab's habit as a standard.

Current status (measured, not projected)

Item	Value
Papers ingested / atomic notes	4 / **80**
Anchor (quote → page) locate rate	**99.7% – 100%**
Generated tools with smoke tests	**5**
Skills	1 approved + 4 drafts awaiting human review
Automated tests	**378 passing, 0 failing**
Code	12,404 lines source + 5,510 lines tests, 21 design documents
Dependencies	2 runtime (`zod` , `pdfjs-dist`), 0 agent frameworks

Two different sample bases behind these numbers: the page-number observation covers **two papers (38 claims)**; the 99.7%–100% locate rate is measured over **all 4 papers / 80 notes**.

Reproduce in one command:

```
npm install && npm run verify        # typecheck + rule checks + 378 tests
node src/main.ts web                 # three-panel UI at http://127.0.0.1:7777
KNOWLEDGE_DIR=fixtures/knowledge node src/main.ts web   # panels preloaded, no PDF needed
```

Runnable link: what to look at in 30 seconds

https://sciencepal.ai/share/7cc3f00e-1999-4c3d-9c32-e842399ac817

This is a **public replay of one complete run** on SciencePal: a close reading of Transformer ("Attention Is All You Need"), `status=completed`, 101.9 s, 44,292 tokens, 554 events. It was opened in a browser with zero cookies to confirm it needs no login.

Where	What it shows
Top	The **prompt exactly as sent** — we gave a paper title and nothing else, no answers, no outline
Middle	The agent working step by step: extracting atomic claims, quoting verbatim, self-reporting page numbers
Right	The "executed commands" terminal, i.e. what it actually called
Bottom	Artifact list and token accounting; the pager jumps to any page so a quote can be checked in place

> **What this link is, and is not.** The current platform version does not offer a "publicly chattable custom agent" (the front end has no publish action, and the marketplace endpoint returns `Custom agent disabled`). So this is a **publicly viewable replay of the full run**, not a session a visitor can continue. For a live run, see the 3–5 minute demo video (item ③-a); to reproduce locally, use the two commands above — `--replay` works fully offline.

Domain scope

The approach is domain-agnostic: it operates on the structure of scientific writing rather than on a single field's data. Our demonstration corpus is NLP/LLM literature (ReAct, Transformer, BERT, RAG) because those papers are public, well typeset, and let reviewers check every quote themselves. Nothing in the pipeline is specific to machine learning; the same extract → anchor → govern path applies to biology, health and materials papers, where traceability matters even more.

Short version (for a single form field)

If the submission form gives you one description field, paste this instead of the full text above:

> **Science Agent Coworker Hajimi — a research agent that grows reusable assets from papers.** Researchers do not lack papers; they lack knowledge that survives the reading. Conclusions lose their provenance, methods are re-implemented by every group, and hard-won judgement stays tacit. Science Agent Coworker Hajimi turns each paper into three compounding libraries: atomic claims anchored to a verbatim quote and a *verified* page number (across two measured runs, only 1 of 38 model-reported page

> numbers was right while the quotes were locatable 38/38, so pages are reverse-located from the quote — 99.7–100% locate rate); runnable tools extracted from the methods described in papers, each with a smoke test; and seven-section skill files that stay in a review queue until a human approves them. SciencePal performs the close reading inside our main loop as a tool node, and our system governs what may enter the knowledge base. Built for graduate students, PIs and systematic-review teams: 4 papers, 80 notes, 5 tools, 378 passing tests, zero agent frameworks.